

# UNIT IV

**Dr. K. K. THYAGHARAJAN**  
**Professor & Dean (Research)**  
**R.M.D. Engineering College**

Scan the QR code and it will provide the link to all videos. It is a trusted link and safe.



<https://www.youtube.com/c/KKTVirtualLearning>

# Linear Regression

**Linear regression:** It is a type of supervised machine learning algorithm that computes the linear relationship between a dependent variable and one or more independent features.

### Types of Linear Regression:

**i) Univariate (or Simple) Linear regression:** Only one independent feature (input / predictor / independent variable) is used

The equation for simple linear regression is  $Y = mX + c$

Where Y is the dependent variable , 'X' is the independent variable,

'c' is the intercept , 'm' is the slope

**ii) Multivariate (or multiple) linear regression:** More than one features are used.

$$Y = c + m_1X_1 + m_2X_2 + \dots + m_nX_n$$

Where Y is the dependent variable,  $X_1, X_2, \dots, X_n$  are the independent variables (features or input variables)

'c' is the intercept and  $m_1, m_2, \dots, m_n$  are the slopes

iii) **Polynomial Regression:** Can be considered as generalized case of linear regression

Single Independent Variable X, polynomial of degree 2

$$Y = b_0 + b_1X + b_2X^2$$

Two variables , polynomial of degree two

$$Y = b_0 + b_1X_1 + b_2X_2 + b_3X_1^2 + b_4X_1X_2 + b_5X_2^2$$

The goal of the regression algorithm is to find the **Best Fit Line equation** that can predict (obtain) the  $y$  value based on the values given to the independent variables.

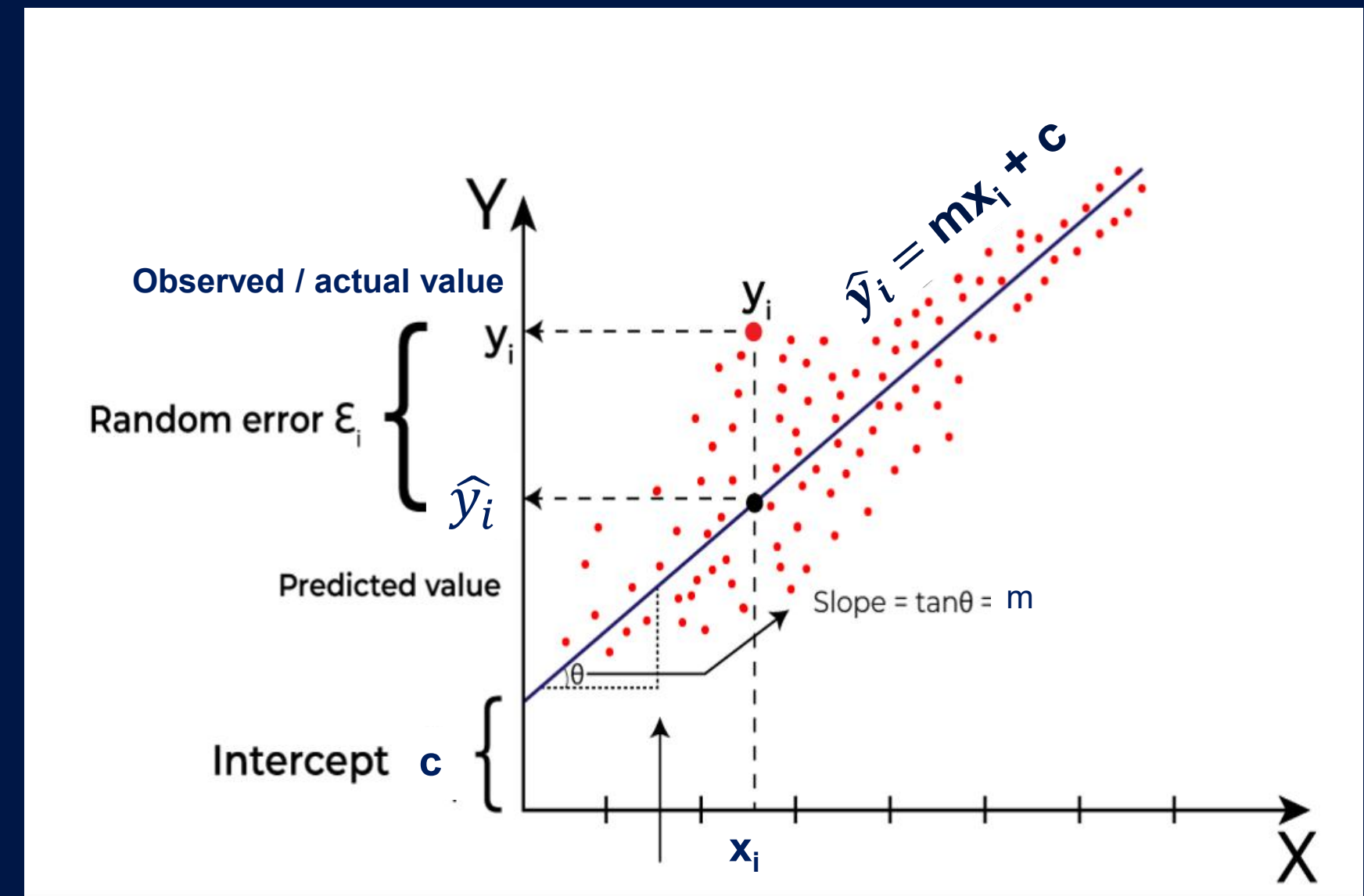
So, a set of records with known 'X' and 'Y' values are used to learn a function and when you want to predict (obtain) ' $y_i$ ' from an unknown  $x_i$ , this learned function is used.  $\hat{y}_i$

If the **error ( $\epsilon_i$ )** between the **predicted and actual values** is kept minimum, then the equation is a Best Fit Line equation.

The **best Fit Line** equation provides a straight line that represents the **linear relationship** between the dependent (Y) and independent (X) variables.

The slope of the line indicates how much the dependent variable changes for a unit change in the independent variable(s).

Since **different** values of the **coefficients** (also called **weights**) result in different regression lines, to find out the best fit line or best coefficients a **cost function (loss function)** is used



To determine the **optimal values** for the **intercept** & the **coefficients** of the linear regression equation and to provide the **best-fit line** for the given data points  $(x_i, y_i)$ , Mean Squared Error (MSE) cost function is employed. It is nothing but the **average of the squared errors** between the **predicted values** and the **actual values**. The **cost function** is also called the **error function** or **loss function**.

If the **predicted value** is denoted as  $\hat{y}$ , then according to the simple linear regression,

$$\hat{y}_i = mx_i + c$$

Where  $x_i$  is the **input value (observed value)** and 'm' & 'c' are coefficients

If  $y_i$  is the **actual output value** of the  $x_i$ , then the **Mean Squared Error (MSE)** cost function is

$$\text{Cost function } (J) = MSE = \frac{1}{n} \sum_{i=0}^n (\hat{y}_i - y_i)^2$$

You will get best-fit line i.e. **optimum values of coefficients**, when the cost function value or **error value** is minimum. Technically we can say that this loss function should reach a global minima.

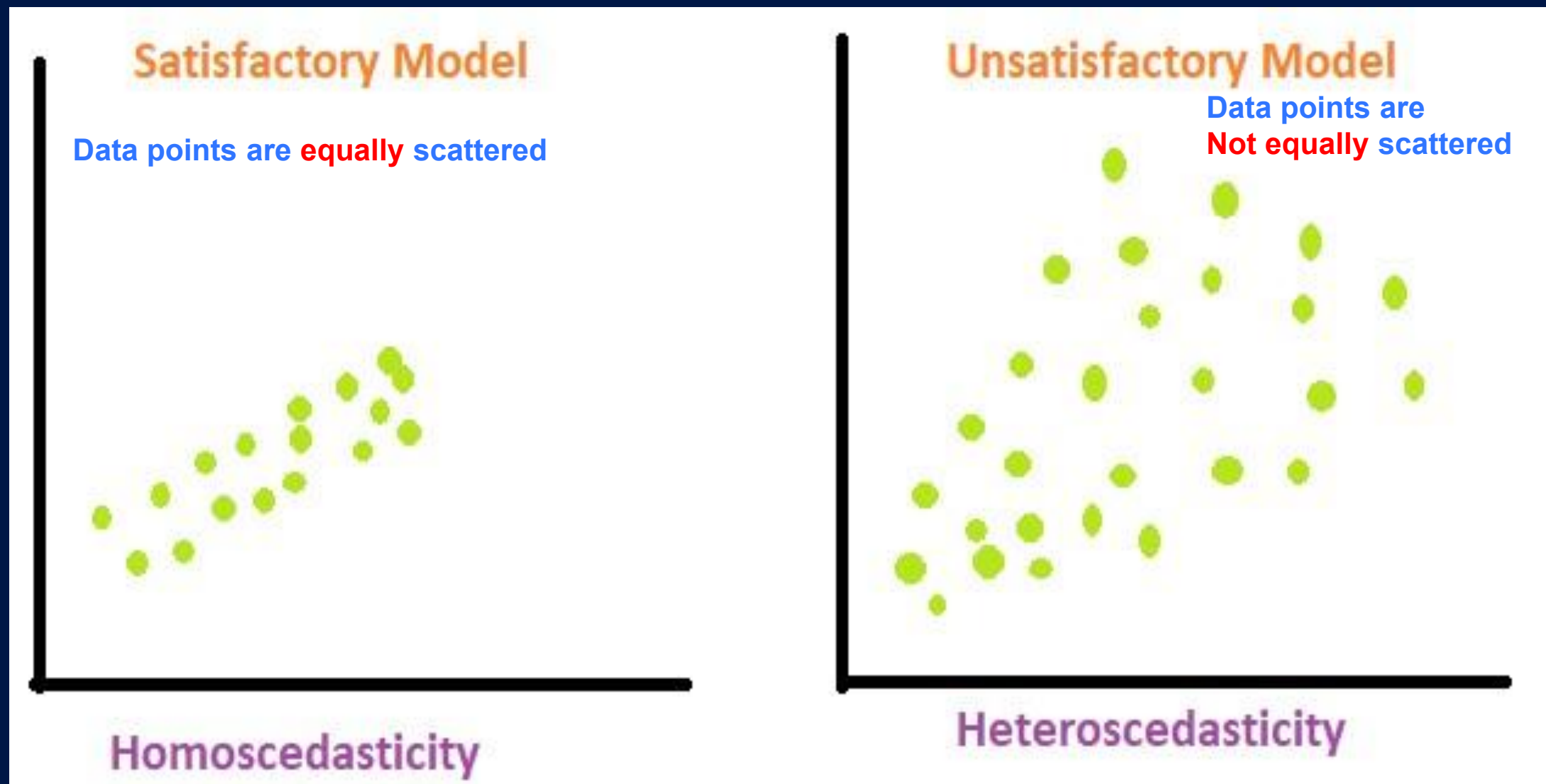
# Assumptions in Linear Regression & Heteroscedasticity

1. The dependent (target) variable is continuous.
2. There is not any relationship between the **independent variables**.
3. There should be a linear relationship between the dependent and explanatory (independent) variables.
4. Error (residuals) should follow a normal distribution.
5. Mean error (residuals) should be zero
6. Variance of the error should be constant.
7. Error should be independently distributed / no autocorrelation.

## Heteroscedasticity in Regression:

One of the assumptions made in linear regression is that the **variance of error** is **constant**. This assumption is known as **Homoscedasticity**.

**Heteroscedasticity** is the condition in which the **variance of error** term or the residual term in a regression model is **not constant**. So, the Estimators are no longer the best or efficient and the T-test and F-Test are no longer valid.



**Identifying heteroscedasticity:**  
It produces either outward opening funnel or outward closing funnel shape in residual plots.

## Identifying Heteroscedasticity Through Statistical Tests:

**Breush – Pagan Test:** It tests whether the **variance of the errors** is changes based on the values of the **independent variables**. In that case, heteroscedasticity is present.

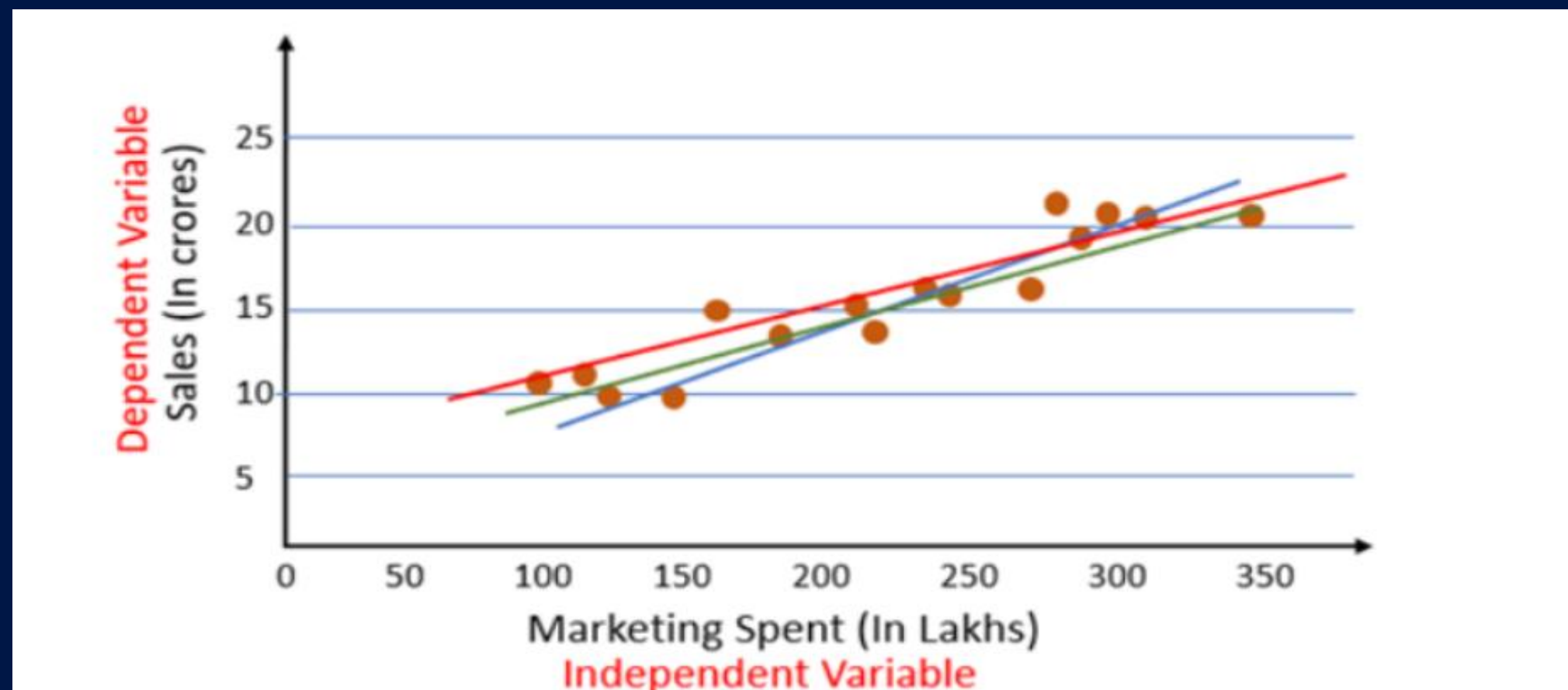
**White test:** This test establishes whether the **variance of the errors** is **constant**. To test for constant variance, one undertakes an auxiliary regression analysis: this regresses the squared residuals from the original regression model onto a set of regressors that contain the original regressors along with their squares and cross-products.

### **Corrections for heteroscedasticity:**

1. Weighted Least Squares method is used
2. Minimum Norm Quadratic Unbiased Estimation (MINQUE) is also used

# **Linear Regression**

## **Gradient Descent Optimization**



In the case of linear regression, (for example,  $y=mx+c$ ), the same data may be fit with different  $m$  and  $c$  values as shown in the figure.

We have to find out the best fit i.e. we have to find out  $m$  and  $c$  values which produce minimum average error.

Gradient descent is an optimization method that uses the gradient of the function to find the optimum value (minimum / maximum) of that function. In linear regression, the parameters  $m$  and  $c$  are to be optimized such that the Mean Square Error (MSE) is minimum. This is also called the cost function ( $J$ ) or loss function.

$$\text{Cost function } (J) = \text{MSE} = \frac{1}{n} \sum_{i=0}^n (\hat{y}_i - y_i)^2$$

This error function should be minimized by choosing optimum values for  $m$  and  $c$ . For this, first we have to find out the derivative of  $J$  w.r.t  $m$  and  $c$

$$MSE = J = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

$\uparrow$  predicted       $\leftarrow$  actual

Where  $\hat{y}_i = mx_i + c$  Substitute this in the above equation, we get

$$J = \frac{1}{n} \sum_{i=1}^n \{(mx_i + c) - y_i\}^2$$

To find out the minimum value of J, differentiate it.

$$\begin{aligned} \frac{\partial J}{\partial m} = D_m &= \frac{1}{n} \sum_{i=1}^n 2(mx_i + c - y_i)(x_i + 0 - 0) \\ &= \frac{1}{n} \sum_{i=1}^n 2x_i(mx_i + c - y_i) \end{aligned}$$

$$\begin{aligned} \frac{\partial J}{\partial c} = D_c &= \frac{1}{n} \sum_{i=1}^n 2(mx_i + c - y_i)(0 + 1 - 0) \\ &= \frac{1}{n} \sum_{i=1}^n 2(mx_i + c - y_i) \end{aligned}$$

Learning factor  $\nwarrow$   
 Update m and c values based on the average gradient  
 i)  $m = m - L D_m$   
 ii)  $c = c - L D_c$

## Gradient Descent Training and Optimization Algorithm:

i)  $\hat{y}_i = mx_i + c$

ii) Initialize m, c, and Learning factor L

iii) Calculate the error in the prediction

i.e.  $MSE = \frac{1}{n} \sum_{i=0}^n (\hat{y}_i - y_i)^2$

If the error is zero or if it has acceptable minimum value, then the m and c values are optimum values and stop the process.

Otherwise continue with step iv

iv) Calculate the average of the gradients of all points for m i.e.  $D_m$

v) Calculate the average of the gradient of all points for c i.e.  $D_c$

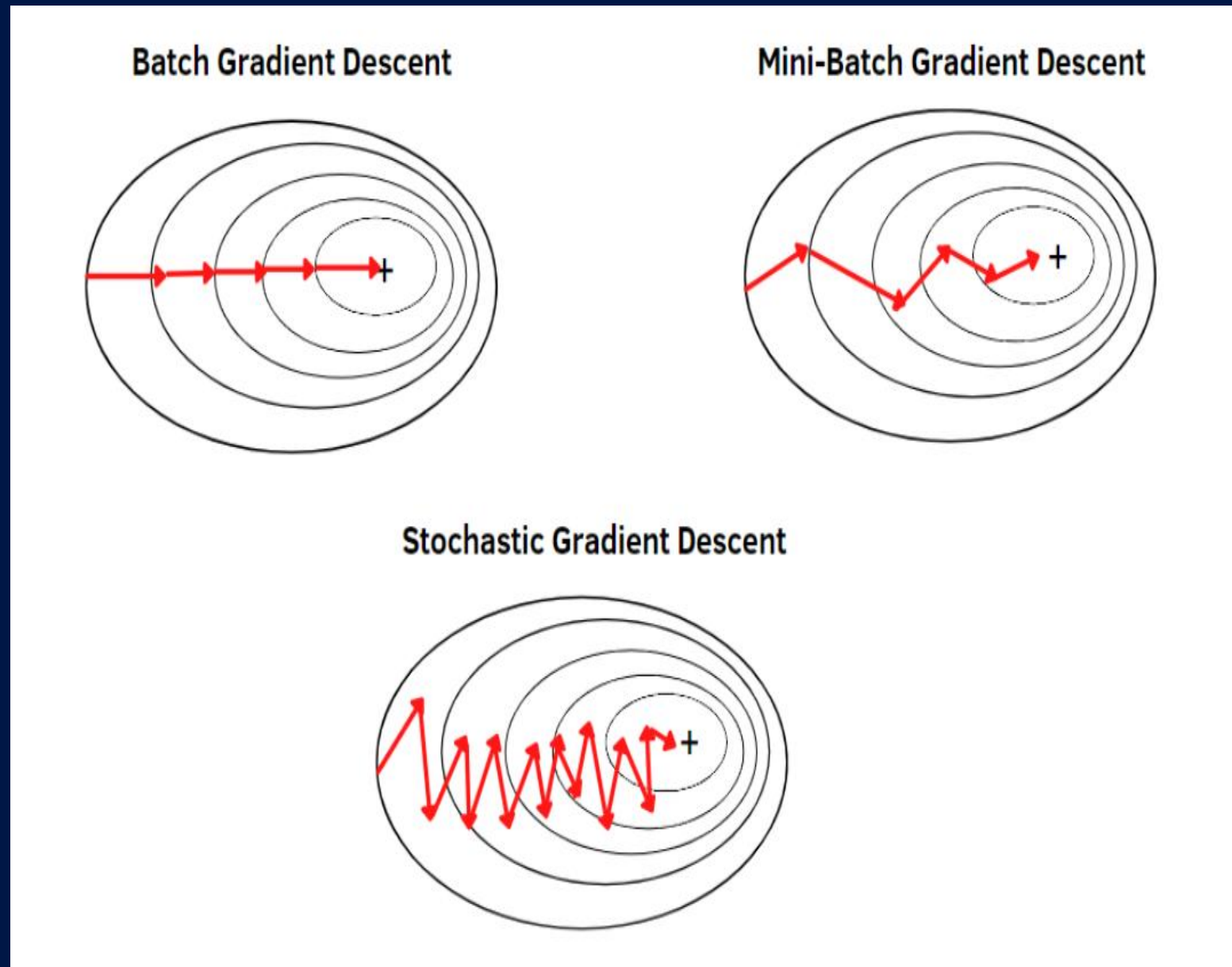
vi) update m and c using

$$m = m - L D_m$$

$$c = c - L D_c$$

Go to step (iii) and repeat the procedure

# Types of Gradient Descent Methods:



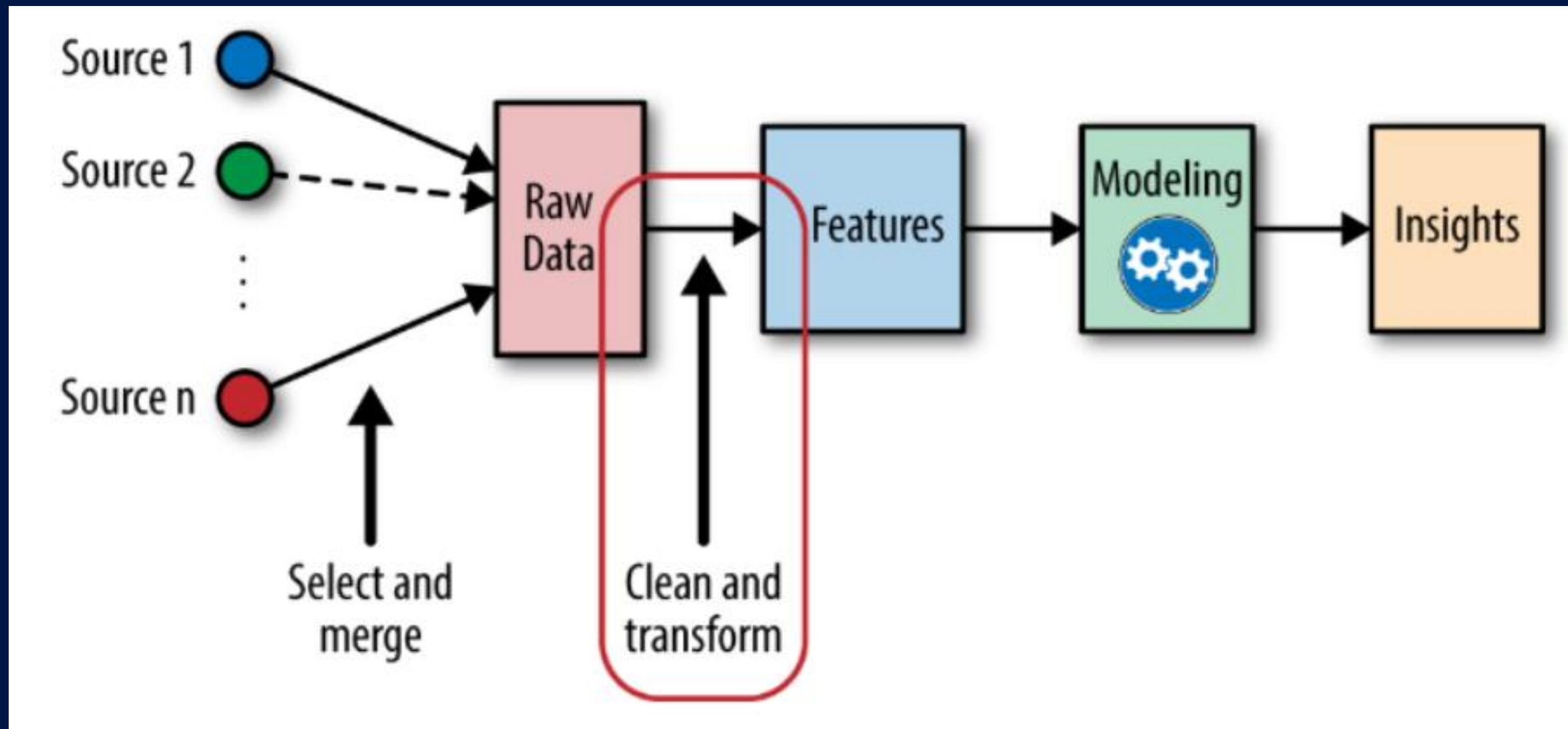
**Batch Gradient Descent:** It is a greedy approach where we have to take the average gradient considering all the data (**one epoch**) in the dataset for each update of gradient (**single batch**).

**Mini-Batch Gradient Descent:** Instead of taking all the data in a single batch, it is divided into mini-batches. Mini-batch Gradient Descent sums up the gradients of this batch of data (**one step**). Batch size is very small compared to the number data in the over all data set. Learning happens in steps for each mini-batch. This may lead to oscillations in learning.

**Stochastic Gradient Descent:** learning happens on every example i.e. for each data  $m$  and  $c$  are updated. Leads to more oscillations in the gradient before reaching the target.

# Feature Engineering

Feature Engineering is the process of extracting and organizing the important features from raw data and transforming them in such a way that it fits the machine learning model.



Feature Engineering encapsulates various data engineering techniques such as selecting **relevant features**, **handling missing data**, **encoding the data**, and **normalizing it**.

Feature Engineering is one of the most crucial tasks and plays a major role in determining the **outcome of a model**. This will ensure that the chosen algorithm can perform effectively and to its optimum capability.

In most cases, Data Scientists deal with data extracted from massive open data sources such as the Internet, surveys, or reviews. This data is crude and is known as raw data.

It may contain missing values, unstructured data, incorrect inputs, and outliers. If we directly use this raw, un-processed data to train our models, we will end up with a model having a very poor efficiency.

Feature engineering is the process of transforming **raw data into features** that are **suitable** for machine learning models.

## **Benefits of Feature Engineering:**

An effective Feature Engineering implies:

**Higher efficiency** of the model

**Easier** Algorithms that **fit** the data

**Easier** for Algorithms to **detect patterns** in the data

**Greater Flexibility** of the features

Well, cleaning up bulks of raw, unstructured, and dirty data may seem like a daunting task, but that is exactly what this guide is all about.

## Need for Feature Engineering in Machine Learning?

**Improve User Experience:** By **adding new features**, we can make the product more intuitive, **efficient**, and user-friendly, which can increase user satisfaction and engagement.

**Competitive Advantage:** By offering unique and **innovative features**, we can differentiate our product from competitors and attract more customers.

**Meet Customer Needs:** By analyzing user **feedback**, **market trends**, and **customer behavior**, we can identify areas where new features could enhance the product's value and meet customer needs.

**Increase Revenue:** A new feature that streamlines the checkout process can **increase sales**, or a feature that provides **additional functionality** could lead to more upsells or cross-sells.

**Future-Proofing:** We can develop features that ensure that the **product remains relevant and useful in the long term**.

## Steps in Feature Engineering:

**Data Cleansing** : also known as **data cleaning** or data scrubbing involves identifying, removing or correcting any **errors or inconsistencies** in the dataset. This step is important to ensure that the data is accurate and reliable.

**Data Transformation** : Transforming the features into a **more suitable representation** for the machine learning model will ensure that the model can effectively learn from the data.

**Feature Extraction** : Extract **relevant features** that may improve the model's accuracy

**Feature Selection**: Feature selection involves **selecting the most relevant features** from the dataset for use in machine learning. This can include techniques like **correlation analysis, mutual information, and stepwise regression**.

**Feature Iteration** : Feature iteration involves **refining and improving the features** based on the **performance** of the machine learning model. This can include techniques like adding new features, **removing redundant features**, and transforming features in different ways

# Techniques Used in Feature Engineering

There are various techniques that can be used in feature engineering to create new features by combining or transforming the existing ones. The following are some of the commonly used feature engineering techniques:

**One-Hot Encoding:** This technique is used to transform categorical variables into numerical values that can be used by machine learning models.

In this technique, each category is transformed into a binary value indicating its presence or absence.

For example, consider a categorical variable “Colour” with three categories: Red, Green, and Blue. One-hot encoding would transform this variable into three binary variables: Colour\_Red, Colour\_Green, and Colour\_Blue, where the value of each variable would be 1 if the corresponding category is present and 0 otherwise.

**Scaling :** The most common scaling techniques are **standardization** and **normalization**. Standardization scales the variable so that it has zero mean and unit variance. Normalization scales the variable so that it has a range of values between 0 and 1.

# Techniques Used in Feature Engineering Contd. ...

**Binning** : This technique is used to transform continuous variables into categorical variables. In this technique, the range of values of the continuous variable is divided into several bins, and each bin is assigned a categorical value. For example, consider a continuous variable “Age” with values ranging from 18 to 80. Binning would divide this variable into several age groups such as 18-25, 26-35, 36-50, and 51-80, and assign a categorical value to each age group.

**Feature Split:** Feature splitting is a powerful technique used in feature engineering to improve the performance of machine learning models. It involves dividing single features into multiple sub-features or groups based on specific criteria. This process unlocks valuable insights and enhances the model’s ability to capture complex relationships and patterns within the data.

**Text Data Preprocessing** : Text preprocessing involves removing stop words, stemming, lemmatization, and vectorization. Stop words are common words that do not add much meaning to the text, such as “the” and “and”. Stemming involves reducing words to their root form, such as converting “running” to “run”. Lemmatization is similar to stemming, but it reduces words to their base form, such as converting “running” to “run”. Vectorization involves transforming text data into numerical vectors that can be used by machine learning models.

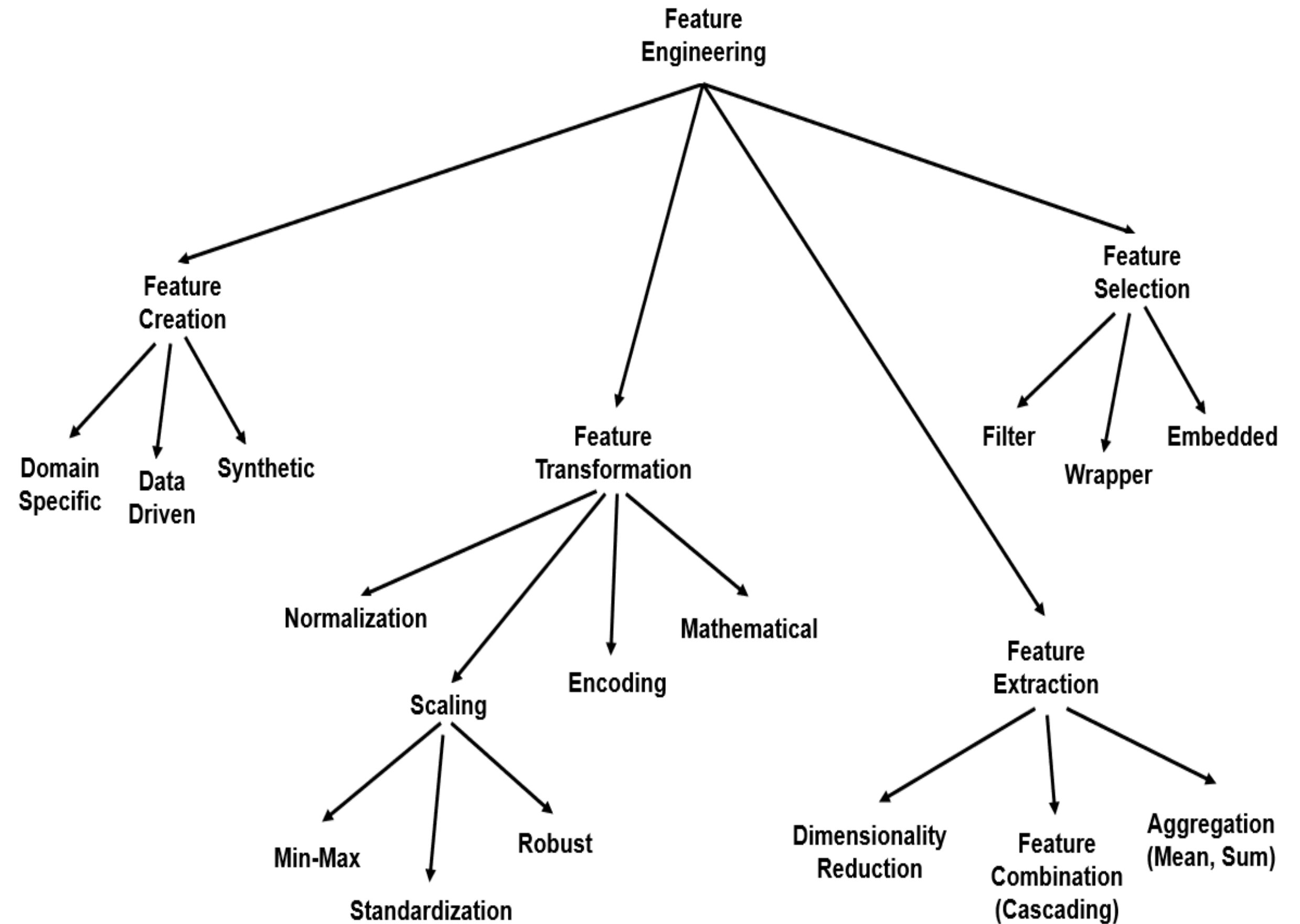
# Processes Involved in Feature Engineering

Feature engineering in Machine learning consists of mainly 5 processes:

Feature Creation, Feature Transformation, Feature Selection, and Feature Scaling.

Finding the best combination of features for a given problem requires experimentation and it is an iterative process.

The success of a machine learning model largely depends on the quality of the features used in the model.



# 1. Feature Creation:

Feature Creation is the process of generating new features based on domain knowledge or by observing patterns in the data.

## Types of Feature Creation:

**Domain-Specific:** Creating new features based on domain knowledge such as business rules or industry standards.

**Data-Driven:** Creating new features by observing patterns in the data, such as calculating **aggregations** or creating **interaction features**.

**Synthetic:** Generating new features by **combining existing features** or synthesizing new data points.

## **Why Feature Creation is Needed?**

**Improves Model Performance:** By providing additional and more relevant information to the model, feature creation can increase the accuracy and precision of the model.

**Increases Model Robustness:** By adding additional features, the model can become more robust to outliers and other anomalies.

**Improves Model Interpretability:** By creating new features, it can be easier to understand the model's predictions.

**Increases Model Flexibility:** By adding new features, the model can be made more flexible to handle different types of data.

## 2. Feature Transformation

Feature Transformation is the process of transforming the features into a **more suitable representation** for the machine learning model. This is done to ensure that the model can effectively learn from the data.

### Types of Feature Transformation:

**Normalization:** Rescaling the features to have a similar range, such as between 0 and 1, to prevent some features from dominating others.

**Scaling:** Scaling is a technique used to transform numerical variables to have a similar scale, so that they can be compared more easily. Rescaling the features to have a similar scale, such as having a standard deviation of 1, to make sure the model considers all features equally.

**Encoding:** Transforming categorical features into a numerical representation. Examples are **one-hot encoding** and **label encoding**. (Refer Unit 2)

**Mathematical:** Transforming the features using mathematical operations to change the distribution or scale of the features. Examples are **logarithmic**, **square root**, and **reciprocal transformations**. (Refer Unit 3)

## Why Feature Transformation is Needed?

**Improves Model Performance:** By transforming the features into a more suitable representation, the model can learn more **meaningful patterns** in the data.

**Increases Model Robustness:** Transforming the features can make the model more robust to **outliers** and other anomalies.

**Improves Computational Efficiency:** Transforming may reduce the computational resources.

**Improves Model Interpretability:** Transforming can help understanding the model's predictions easily.

## 3. Feature Extraction

Feature Extraction is the process of creating / extracting **more relevant new features** from existing ones. This is done by transforming, combining, or aggregating the existing features.

### Types of Feature Extraction:

**Dimensionality Reduction:** Reducing the number of features by transforming the data into a lower-dimensional space while retaining important information. Examples are PCA and t-SNE.

**Feature Combination:** Combining two or more existing features to create a new one. For example, the interaction between two features.

**Feature Aggregation:** Aggregating features to create a new one. For example, calculating the mean, sum, or count of a set of features.

### Why Feature Extraction is Needed?

**Improves Model Performance:** By selecting or extracting more relevant features, the model can learn more meaningful patterns in the data.

**Reduces Overfitting:** By reducing the dimensionality of the data, the model is less likely to overfit the training data.

**Improves Computational Efficiency:** The transformed new features often require fewer computational resources.

**Improves Model Interpretability:** By creating new features, it can be easier to understand the model's predictions.

## 4. Feature Selection

Feature Selection is the process of selecting a subset of relevant features from the dataset to be used in a machine-learning model.

### Types of Feature Selection:

**Filter Method:** Based on the statistical measure of the relationship between the feature and the target variable, features with a high correlation are selected.

**Wrapper Method:** The feature subset that results in the best performance is selected.

**Embedded Method:** Features are selected as part of the training process of the machine learning algorithm.

### Why Feature Selection is Needed?

**Reduces Overfitting:** By using only the most relevant features, the model can generalize better to new data.

**Improves Model Performance:** Selecting the right features can improve the accuracy, precision, and recall of the model.

**Decreases Computational Costs:** A smaller number of features requires less computation and storage resources.

**Improves Interpretability:** By reducing the number of features, it is easier to understand and interpret the results of the model.

# 5. Feature Scaling

Feature Scaling is the process of transforming the features so that they have a similar scale.

### a) Normalization:

This is the simplest method of scaling where the features are rescaled to a given range. It is of two types

i) Min-Max Normalization: Also called as Min-Max Scale, this transforms data i.e. each feature individually to fall **between zero and one**. This method **preserves the shape of the original distribution**. The transformation is given by the formula.

$$\text{normalized value} = \frac{(x - \min(x))}{(\max(x) - \min(x))}$$

ii) Robust Scaler: As per sklearn, this type of normalization transforms the vector by subtracting the **median** and scales the data between the **1st quartile (25% value)** and **3rd quartile (75% value)**.

b) **Standardization:** This is the most common form of feature scaling applied to many learning algorithms. First, we calculate the distribution mean and standard deviation for each feature and calculate the new data point by using the formula.

$$x' = \frac{x - \bar{x}}{\sigma}$$

Here,  $\sigma$  is the standard deviation of the feature vector, and  $\bar{x}$  is the average of the feature vector.

Standardization results in a distribution that has a standard deviation of 1 and a mean of 0. Examples of algorithms that use Standardization are Support Vector Machine (SVM), logistics regression and neural networks. The Python code to implement this scaling is given below

Example: An algorithm that uses K-nearest neighbors (KNN).

Python code:

```
scaler = MinMaxScaler()  
x_minmax = scaler.fit_transform(x)
```

Python code for robust scaling:

```
from sklearn.preprocessing import RobustScaler  
transformer = RobustScaler().fit(X)  
transformer.transform(X)
```

```
from sklearn.preprocessing import StandardScaler  
sc = StandardScaler()  
X_train[:, :] = sc.fit_transform(X_train[:, :])
```

### Why Feature Scaling is Needed?

**Improves Model Performance:** By transforming the features to have a similar scale, the model can learn from all features equally and avoid being dominated by a few large features.

**Increases Model Robustness:** By transforming the features to be **robust to outliers**, the model can become more robust to anomalies.

**Improves Computational Efficiency:** Many machine learning algorithms, such as k-nearest neighbors, are sensitive to the scale of the features and perform better with scaled features.

**Improves Model Interpretability:** By transforming the features to have a similar scale, it can be easier to understand the model's predictions.

## Where to use Feature Scaling?

Min-Max Scaler is used to have a **light touch** on features

Normalization is used when the distribution of data **does not follow Gaussian distribution**.

Robust Scaler is used to reduce the **influence of outliers**.

Standardization is applied when **Gaussian distribution is followed** e.g. Principal Component Analysis (PCA).

For **K-Means Clustering**, the Euclidean distance is important, so **Feature Scaling** makes a huge impact.

Feature scaling also makes a huge impact on any algorithms that rely on **gradients**, such as linear models that are fitted by **minimizing loss with Gradient Descent**.

Some feature selection methods will depend on the scale of the data, in which case it seems best to scale beforehand.

A **tree-based model won't suffer from unscaled data**, because scale doesn't affect them at all.

**Disadvantage:** Both normalization and standardization are **sensitive to outliers**.

# Why is feature scaling needed?

Feature scaling is done during **Data Preprocessing** to adjust the features to a common scale.

Feature scaling is similar to the database normalization method and is used to normalize the range of **independent** (features) data.

Feature scaling helps in not drifting towards high-range input values i.e. it resolves (or) reduces the disparity between the features of data like **high bias**, and values varying from each other on a broad scale (high values).

It brings the value / magnitude of the numbers close to each other within a range for easy comparison. This makes the models interpret the features easier, bring them into the same scale and make them perform better.

Generally, various **Machine Learning models don't generalize** well on data when the data has **high-scale variance**. The same data can be bad for one model, but great for another.

You can have the best model crafted for any sort of problem. But if you feed it garbage, it'll spew out garbage. It's worth noting that **"garbage" doesn't refer to random data**. It's a **harsh label** we attach to any data that doesn't allow the model to do its best.

# Performing Data Scaling Before Data Split

## Disadvantage:

Performing feature scaling before dataset splitting can introduce **data leakage (from test data)**, as the scaling parameters such as mean and standard deviation are influenced by the test set, leading to over optimistic evaluation of results.

**Data leakage** is a phenomenon that occurs when **information outside of the training data is used to train** a model. It essentially violates the independence of the training data and allows it to be altered by information from test data.

The model's performance would only be limited to the range in which the data is scaled e.g. in the range of  $[0,1]$ . So, the data in different ranges (test data) will make the model fail.

## Advantage:

Scaling the entire dataset before splitting **ensures that the same scaling factors** are applied to both the training and test sets. This **leads to consistency** in the data preprocessing pipeline. It allows you to explore the entire dataset during exploratory data analysis (EDA).

## Performing Data Scaling After Data Split:

Feature scaling is performed only on the training set. This ensures that there is no **data leakage** from the **test set**.

### **Advantage:**

Parameters obtained during the normalization/scaling of training data can be used to normalize the test data and also change it back to the original scale when showing the output/predictions.

This approach provides a more realistic evaluation of the model's performance since it mimics real-world scenarios where new, unseen data requires scaling.

### **Disadvantages**

It may cause a slight shift in the distribution of the training set features, potentially leading to a decrease in model performance.

# Feature Engineering Tools

**1. Feature tools** : This is a **Python library** that enables automatic feature engineering for **structured data**. It can extract features from **multiple tables**, including relational databases and CSV files, and generate new features based on **user-defined primitives**. Some of its features include:

Automated feature engineering using **machine learning** algorithms.

Support for handling time-dependent data.

Integration with popular Python libraries, such as pandas and sci-kit-learn.

Visualization tools for exploring and analyzing the generated features.

Extensive documentation and tutorials for getting started.

**2. TPOT (Tree-based Pipeline Optimization Tool)**: This uses **genetic programming** to search for the best combination of features and machine learning algorithms for a given dataset. Some of its features include:

Automatic feature selection and transformation.

Support for multiple types of machine learning models, including regression, classification, and clustering.

Ability to **handle missing** data and **categorical variables**.

Integration with popular Python libraries, such as scikit-learn and pandas.

Interactive visualization of the generated pipelines.

**3. Data Robot :** This is a machine learning automation platform that includes feature engineering as one of its capabilities. It uses **automated machine learning techniques** to generate new features and select the best combination of features and models for a given dataset. Some of its features include:

Automatic feature engineering using machine learning algorithms.

Support for handling time-dependent and text data.

Integration with popular Python libraries, such as pandas and scikit-learn.

Interactive visualization of the generated models and features.

Collaboration tools for teams working on machine learning projects.

**4. Alteryx :** Alteryx is a data preparation and automation tool that includes feature engineering as one of its features. It provides **a visual interface for creating data pipelines** that can extract, transform, and generate features from multiple data sources. Some of its features include:

Support for **handling structured and unstructured data**.

Integration with popular data sources, such as Excel and databases.

Pre-built tools for feature extraction and transformation.

Support for custom scripting and code integration.

Collaboration and sharing tools for teams working on data projects.

**5. H2O.ai** : This is an open-source machine learning platform that includes feature engineering as one of its capabilities. It provides a range of automated feature engineering techniques, such as **feature scaling, imputation, and encoding**, as well as manual feature engineering capabilities for more advanced users. Some of its features include:

Automatic and manual feature engineering options.

Support for structured and unstructured data, including text and image data.

Integration with popular data sources, such as CSV files and databases.

Interactive visualization of the generated features and models.

Collaboration and sharing tools for teams working on machine learning projects.

# Building a Predictive Model

# Building a Predictive Model

Predictive modeling is the use of statistical models to make predictions about the future from past data. This can be used in industries for the following situations

When we talk about predictive models, we are talking either about a regression model (continuous output) or a classification model (nominal or binary output).

Identifying content-violating posts for a social media site

Trying to predict the future value of a stock in finance

Estimating the likelihood that a policy gets claimed in insurance

Predicting the effectiveness of an advertising campaign

## Steps to Build a Predictive Model

### 1. Collect and Organize the Dataset

The first step to constructing a predictive model is to gather the data for which you want to build your model. These data show how the variables influence the outcome you're trying to predict.

You can get data by downloading it from a website or scraping it from the web.

The company may provide a database made in software like MySQL, MongoDB, or Neo4j to query your data.

# Building a Predictive Model

**2. Clean the Dataset:** Often, a dataset won't be “clean” when you first open it up. Some examples include:

Different spellings for column values (i.e., having “**Texas**” and “**TX**” in a “state” column)

Incorrect decimal points or units (i.e., price information in **cents** rather than **dollars**)

Missing data (i.e. having values labeled as “**NULL**”, “**NA**”, or “**NaN**”)

Since there are infinite ways that someone may wrongly display data, there's no “one size fits all” approach to resolve these issues.

The optimal strategy is to think critically about addressing each problem independently.

Some best practices are:

**Never process “numerical”** data that are **descriptive** (i.e. **zip codes**) as an actual number.

It's generally better to **drop missing data** rather than replace it with **zero** or the **average**.

Ensure there's exactly one way to label a category (so nothing like “**small**” and “**Small**” and “**S**”).

# Building a Predictive Model

## 3. Choose a Methodology / Algorithm

The next step is to choose the model you want to use to generate your predictions. There are hundreds of models to choose from, and one way to narrow them down is to determine if a **regression or classification** model suits the situation.

Regression models output a numerical estimation of the value of a specific variable, i.e. the price of a stock. Classification models output a statement of what “class” or “category” a data point belongs to, i.e. if someone defaults on a loan or not.

To choose between using a regression or classification model, ask yourself what you want the output to be. If the output is a number, go with a regression model. If it’s a category, go with a classification model.

As for what specific model to pick, feel free to review the **sklearn** user guide for high-level overviews about different models.

# Building a Predictive Model

## 4. Build the Model

The final step is to build and fine-tune the model. You can follow a standard format for training the model.

No matter what programming language you use, there will be **language-specific packages, functions**, and syntax you'll need to use. Python is no different.

Data scientists can generally conduct predictive modeling in Python through the **NumPy, pandas, and scikit-learn** packages.

Pandas and NumPy can help you load and manipulate data, while scikit-learn lets you build the predictive model. A more specialized model may use additional packages.

# Evaluation metrics for Linear Regression Model

Machine learning model should not be biased for a specific dataset due to over fitting or under fitting, it should be **generalized** to have 100 percent efficiency.

It is necessary to obtain the accuracy on training data, But it is also important to get a genuine and approximate result on unseen data otherwise Model is of no use.

So to build and deploy a generalized model, we have **to evaluate the model** on different metrics to better optimize the performance, fine-tune it, and obtain a better result.

### **Mean Absolute Error (MAE):**

Linear regression creates a best-fit line between the inputs and outputs.

The absolute difference between actual output and output values predicted by the fitted line. This is called error or loss.

Find out the absolute errors for all data in dataset and then compute the mean error i.e. sum all the errors and divide them by the total number of observations this is MAE.

We aim to get a minimum MAE

MAE is most robust to outliers

At points where  $\hat{y}_i = y_i$ , the derivative is not defined, which leads to a lack of differentiability at those points in the MAE function. Therefore, optimization methods that rely on derivatives, such as gradient descent, may encounter difficulties when using MAE as the loss function. This is one reason why alternative loss functions such as Mean Squared Error (MSE) are often preferred in certain scenarios.

The diagram illustrates the Mean Absolute Error (MAE) formula:  $MAE = \frac{1}{N} \sum |Y - \hat{Y}|$ . Annotations include: 'Divide by total Number of Data Points' pointing to  $\frac{1}{N}$ ; 'Actual Output' pointing to  $Y$ ; 'Predicted Output' pointing to  $\hat{Y}$ ; 'Sum Of' pointing to the summation symbol  $\sum$ ; and 'Absolute Value of residual' pointing to the absolute value bars  $|Y - \hat{Y}|$ . A small blue icon is visible in the top right corner of the diagram.

## Mean Squared Error (MSE)

MSE is the most used and very simple metric with a little bit of change in mean absolute error.

Mean squared error states that finding the squared difference between actual and predicted value.

We perform squared to avoid the cancellation of negative terms and it is the benefit of MSE.

$$MSE = \frac{1}{n} \sum_{i=0}^n (y_{i \text{ actual}} - y_{i \text{ predicted}})^2$$

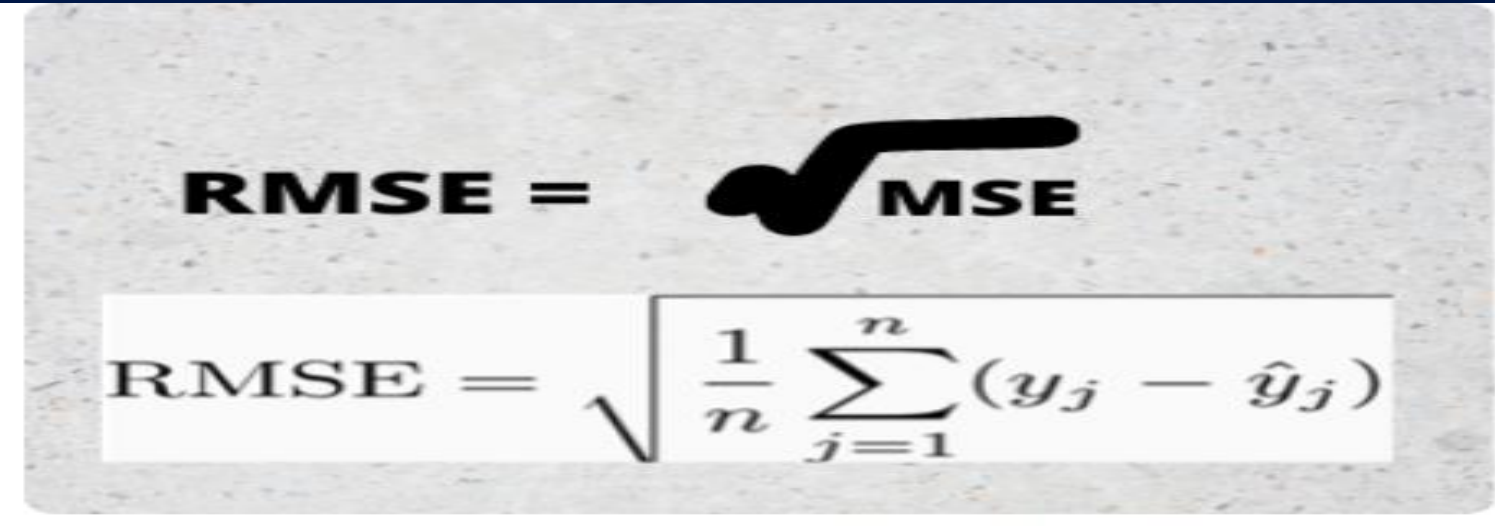
**Advantage:** The graph of MSE is differentiable, so you can easily use it as a loss function.

**Disadvantage:** If you have outliers in the dataset then it penalizes the outliers most and the calculated MSE is bigger. So, in short, It is **not Robust** to **outliers** which were an advantage in MAE

### Root Mean Squared Error (RMSE):

**Advantage:** The output value you get is in the same unit as the required output variable which makes interpretation of loss easy.

**Disadvantage:** It is not that robust to outliers as compared to MAE



The image shows a handwritten formula for RMSE on a piece of paper. The top part shows 'RMSE =' followed by a large square root symbol and 'MSE'. The bottom part shows the full mathematical formula: 
$$RMSE = \sqrt{\frac{1}{n} \sum_{j=1}^n (y_j - \hat{y}_j)^2}$$

### Root Mean Squared Log Error (RMSLE):

When the output is varying in a large scale in the case of RMSE, we take the log of calculated RMSE error and it is called RMSLE.

Taking the log of the RMSE metric **slows down** the scale of error.

## Coefficient of Determination - R Squared (R<sup>2</sup>):

**R<sup>2</sup> score is a metric** that tells the performance of your model, **not the loss**. R<sup>2</sup> squared is also known as Goodness of fit

The coefficient of determination (R<sup>2</sup>) measures how well a statistical model predicts an outcome.

R<sup>2</sup> squared calculates **how much the regression line is better than a mean line**.

With the help of R squared we have a **baseline model** to compare a model which none of the other metrics provides.

The same we have in classification problems which we call a **threshold** which is fixed at 0.5.

MAE and MSE depend on the context whereas the R2 score is independent of context.

$$\mathbf{R^2\ Squared = 1 - \frac{SSr}{RSSm}}$$

**SSr = Squared sum error of regression line**

**SSm = Squared sum error of mean line**

## Adjusted R Squared :

The disadvantage of the  $R^2$  score is while adding new features to data the  $R^2$  score starts **increasing** or remains constant but it never decreases because it assumes that while adding more data, the variance of data increases.

In  $R_a^2$  when new irrelevant features are added, the denominator decreases and the numerator  $(n-1)$  remains constant. When we subtract this from one,  $R_a^2$  will decrease.

If we add a relevant feature then the  $R^2$  score will increase and  $1-R^2$  will decrease heavily and the denominator will also decrease and the complete term decreases. On subtracting this from one,  $R_a^2$  score increases.

$$R_a^2 = 1 - \left[ \left( \frac{n-1}{n-k-1} \right) \times (1 - R^2) \right]$$

where:

$n$  = number of observations

$k$  = number of independent variables

$R_a^2$  = adjusted  $R^2$

# Logistic Regression

Logistic regression is used for **classification** problems.

If the data are classified into only two groups, such as Class 1 & Class 2, True & False, Yes & No, or 0 & 1, or Group 1 & Group 2, then it is called **binary classification**.

If there are more than two classes, then it is called **multi-class classification**.

A sigmoid function predicts the output  $\hat{y}_i$ , and a Binary Cross-Entropy Loss (Log Loss) is used to minimize the prediction error.

$$\hat{y}_i = \text{sig}(x) = \sigma(x) = \frac{1}{1 + e^{-x}}$$

when  $x = -\infty$   $\text{sig}(x) = 0$

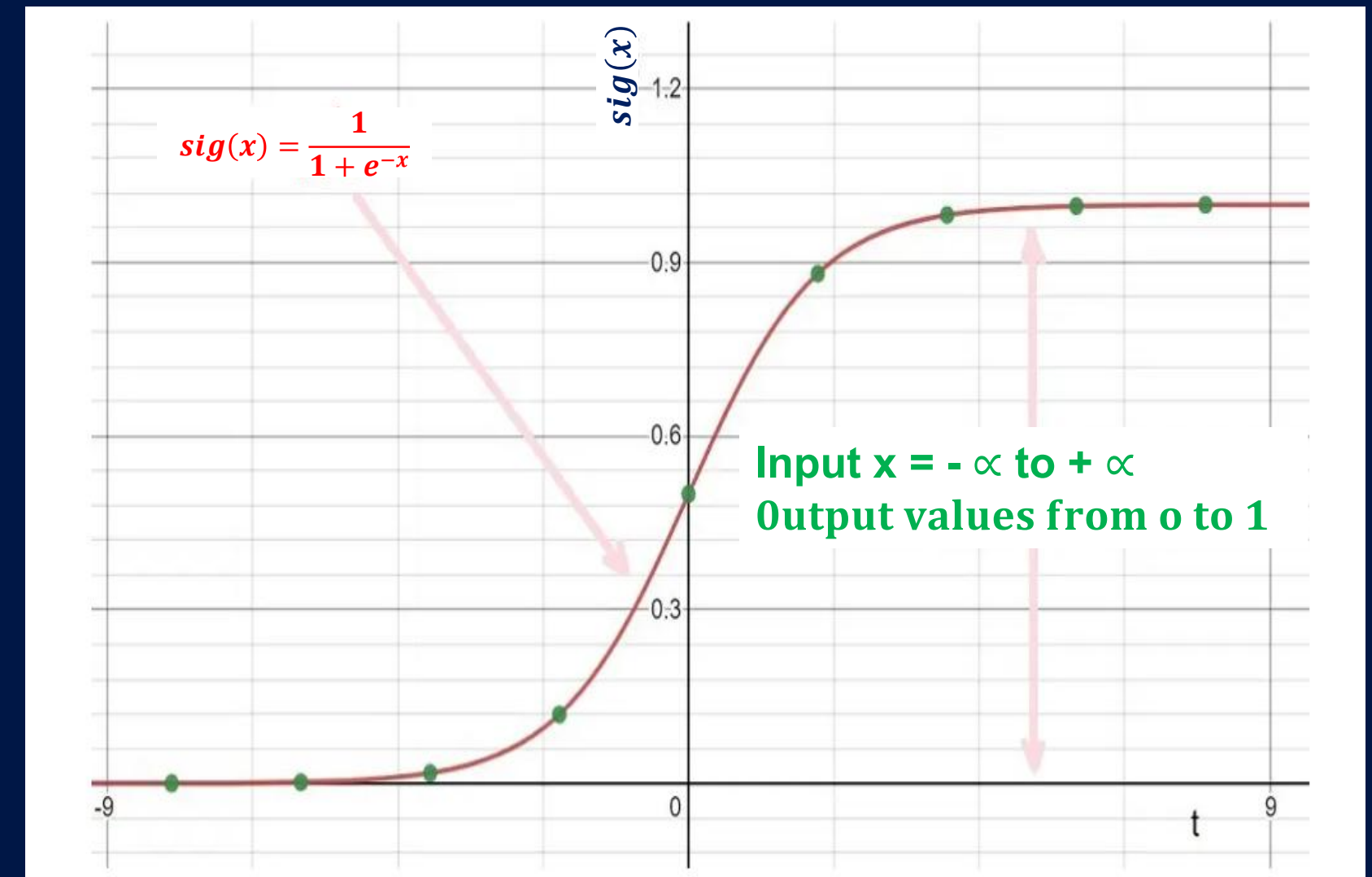
when  $x = 0$  (0.5 is the Threshold)  $\text{sig}(x) = 0.5$

when  $x = +\infty$   $\text{sig}(x) = 1$

So, sigmoid function takes independent variable(s) as input ( $x_i$ ) and produces a probability value  $p(x_i)$  between 0 and 1 as output.  $p(x_i) = \text{sig}(x_i)$

Probability for being data  $x_i$  in the class is given by  $p(y = 1) = \text{sig}(x_i) \rightarrow \text{Yes}$

Probability for not being in the class is given by  $p(y = 0) = 1 - \text{sig}(x_i) \rightarrow \text{No}$



# Logistic Regression For Binary Classification

Instead of either 0 or 1 for binary classification, the Logistic Regression provides values between 0 and 1. Hence it is used to predict the probability of categorical variable.

For binary classification problem, where the target variable  $y$  is either 0 or 1, and the **Binary Cross-Entropy loss function (Log Loss)** is used to estimate the error between the actual value and predicted value.

$$\text{Logloss} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] \quad \dots (1)$$

Where

$N$  = Total number of samples

$y_i$  = Actual class label 0 or 1

$\hat{y}_i$  = predicted probability between 0 and 1

$$\begin{aligned} \text{So, the Logloss} &= -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + \log(1 - \hat{y}_i) - y_i \log(1 - \hat{y}_i)] \\ &= -\frac{1}{N} \sum_{i=1}^N \left[ y_i \log\left(\frac{\hat{y}_i}{(1 - \hat{y}_i)}\right) + \log(1 - \hat{y}_i) \right] \quad \dots (2) \end{aligned}$$

Substitute  $\hat{y}_i = sig(x_i) = \frac{1}{1+e^{-x_i}} = p(x_i)$

$$\frac{\hat{y}_i}{(1 - \hat{y}_i)} = \frac{\frac{1}{1 + e^{-x_i}}}{\left(1 - \frac{1}{1 + e^{-x_i}}\right)} = \frac{\frac{1}{1 + e^{-x_i}}}{\frac{1 + e^{-x_i} - 1}{1 + e^{-x_i}}} = \frac{1}{e^{-x_i}} = e^{x_i} \quad \dots (3)$$

$$(1 - \hat{y}_i) = 1 - \frac{1}{1 + e^{-x_i}} = \frac{1 + e^{-x_i} - 1}{1 + e^{-x_i}} = \frac{e^{-x_i}}{1 + e^{-x_i}} = \frac{1}{\frac{1}{e^{-x_i}} + \frac{e^{-x_i}}{e^{-x_i}}} = \frac{1}{\frac{1}{e^{-x_i}} + 1} = \frac{1}{1 + e^{x_i}} \quad \dots (4)$$

Substitute 3& 4 in equation (2)

$$Log Loss = -\frac{1}{N} \sum_{i=1}^N \left[ y_i \log(e^{x_i}) + \log\left(\frac{1}{1 + e^{x_i}}\right) \right] = -\frac{1}{N} \sum_{i=1}^N \left[ (y_i \times x_i) + \log\left(\frac{1}{1 + e^{x_i}}\right) \right] \quad \dots (5)$$

$$\log\left(\frac{1}{1 + e^{x_i}}\right) = \log(1 + e^{x_i})^{-1} = -\log(1 + e^{x_i})$$

Substituting this in equation 5, we get,

$$\mathbf{Log Loss} = -\frac{1}{N} \sum_{i=1}^N (y_i \times x_i) + \frac{1}{N} \sum_{i=1}^N \log(1 + e^{x_i}) \quad \dots (6)$$

When the input is passes through a weight 'w' , and a bias 'b' added before applying the sigmoid function,

$$x_i = wx_i + b$$

So, equation (5) becomes,

$$\mathbf{Log Loss}(b, w) = \frac{1}{N} \sum_{i=1}^N \log\left(1 + e^{(wx_i + b)}\right) - \frac{1}{N} \sum_{i=1}^N (y_i \times (wx_i + b)) \quad \dots (7)$$

Equation 7, shows that the loss depends on the parameters w and b. We can choose optimum values for w and b such that the loss becomes minimum for the given data

## Logistic Regression For K Classes (Multinomial LR)

The target variable can have 3 or more possible types which are not ordered (i.e. types that have no quantitative significance) like “disease A” vs “disease B” vs “disease C”.

In this case, the softmax function is used in place of the sigmoid function. Softmax function for K classes will be:

In Multinomial Logistic Regression, the output variable can have more than two possible discrete outputs

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

$$P(Y = c | \vec{X} = x) = \frac{e^{w_c \cdot x + b_c}}{\sum_{k=1}^K e^{w_k \cdot x + b_k}}$$

## Type of Logistic Regression:

**Binomial:** There can be only two possible types of dependent variables, such as 0 or 1, Yes or No, etc.

**Multinomial:** There can be three or more possible unordered types of the dependent variable, such as “cat,” “dogs,” or “sheep.”

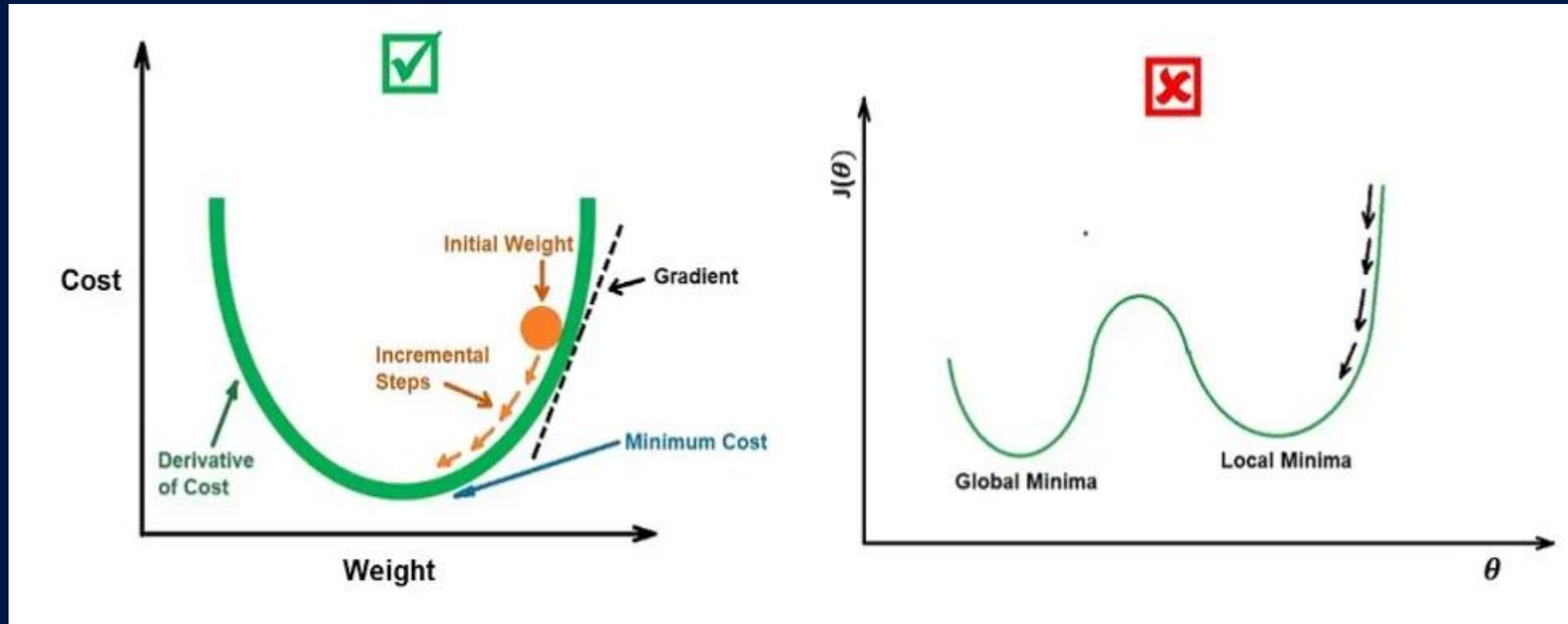
**Ordinal:** There can be three or more possible ordered types of dependent variables, such as “low,” “Medium,” or “High.”

- ❖ The Sigmoid function can map any real value output into a value within 0 and 1.
- ❖ Affected little by outliers.

Algorithms like Logistic Regression, Random Forest, Gradient Boosting, Adaboost, etc., give probability outputs. Converting probability outputs to class output is just a matter of creating a threshold for the probability.

In regression problems, we do not have such inconsistencies in output. The output is always continuous in nature and requires no further treatment

# Logistic Regression



## Assumptions Involved in Implementing Logistic Regression:

**Independent observations:** Each observation is independent of the other, meaning there is **no correlation** between any input variables.

**Binary dependent variables:** It takes the assumption that the dependent variable must be binary or dichotomous, meaning it can take only two values. For more than two categories **SoftMax** functions are used.

**Linearity relationship between independent variables and log odds:** The relationship between the independent variables and the log **odds** (**occurring/not occurring**) of the dependent variable should be linear.

**No outliers:** There should be no outliers in the dataset.

**Large sample size:** The sample size is sufficiently large

## Terminologies involved :

**Independent variables:** The input characteristics or predictor factors or features

**Dependent variable:** The target variable in a logistic regression model, which we are trying to predict.

**Logistic function:** The formula used to represent how the independent and dependent variables relate to one another. The logistic function transforms the input variables into a probability value between 0 and 1, which represents the likelihood of the dependent variable being 1 or 0.

**Odds:** It is the ratio of something occurring to something not occurring. It is different from probability as the probability is the ratio of something occurring to everything that could occur.

$$\frac{\hat{y}_i}{(1 - \hat{y}_i)} = \frac{P(x)}{1 - p(x)}$$

**Log-odds:** The log-odds, also known as the **logit** function, is the natural logarithm of the odds. In logistic regression, the log odds of the dependent variable are modeled as a linear combination of the independent variables and the intercept.

## Terminologies involved Contd. ... :

**Coefficient:** The logistic regression model's estimated parameters, show how the independent and dependent variables relate to one another.

**Intercept:** A constant term in the logistic regression model, which represents the log odds when all independent variables are equal to zero.

**Maximum likelihood estimation:** The method used to estimate the coefficients of the logistic regression model, which maximizes the likelihood of observing the data given the model.

# Evaluation metrics for Regression Models

Evaluation metrics are quantitative measures used to assess the performance and effectiveness of a statistical or machine-learning model.

These metrics provide insights into how well the model is performing and help in comparing different models or algorithms.

When evaluating a machine learning model, it is crucial to assess its predictive ability, generalization capability, and overall quality.

Evaluation metrics provide objective criteria to measure these aspects. The choice of evaluation metrics depends on the specific problem domain, the type of data, and the desired outcome.

### **i) Confusion Matrix**

It is a performance measurement for machine learning classification problems where the output can be two or more classes.

A confusion matrix is an  $N \times N$  matrix, where  $N$  is the number of classes.

It is extremely useful for measuring precision-recall, Specificity, Accuracy,

**True Positive:** You predicted positive, and it's true.

**True Negative:** You predicted negative, and it's true.

**False Positive:** (Type 1 Error): You predicted positive, it's false. Actually it should be negative

**False Negative:** (Type 2 Error): You predicted negative, it's false. Actually it should be positive

**Accuracy:** the proportion of the total number of correct predictions that were correct.

**Positive Predictive Value or Precision:** the proportion of positive cases that were correctly identified.

**Negative Predictive Value:** the proportion of negative cases that were correctly identified.

**Sensitivity or Recall:** the proportion of actual positive cases which are correctly identified.

**Specificity:** the proportion of actual negative cases which are correctly identified.

**Rate:** It is a measuring factor in a confusion matrix. It has also 4 types TPR, FPR, TNR, and FNR.

**FN & FP should be reduced –  
E.g. cancer & advertisement  
applications**

|              |          | Predicted Class                          |                                                          |                                                                            |
|--------------|----------|------------------------------------------|----------------------------------------------------------|----------------------------------------------------------------------------|
|              |          | Positive                                 | Negative                                                 |                                                                            |
| Actual Class | Positive | True Positive (TP)                       | False Negative (FN)<br>Type II Error                     | <b>Sensitivity</b><br>$\frac{TP}{TP + FN}$<br>Recall<br>True Positive Rate |
|              | Negative | False Positive (FP)<br>Type I Error      | True Negative (TN)                                       | <b>Specificity</b><br>$\frac{TN}{TN + FP}$                                 |
|              |          | <b>Precision</b><br>$\frac{TP}{TP + FP}$ | <b>Negative Predictive Value</b><br>$\frac{TN}{TN + FN}$ | <b>Accuracy</b><br>$\frac{TP + TN}{TP + TN + FP + FN}$                     |

**For High recall, FN should be reduced**

**For High Precision, FP should be reduced.**

Logistic regression becomes a classification technique only when a decision threshold is brought into the picture.

The setting of the threshold value is a very important aspect of Logistic regression and is dependent on the classification problem itself.

The decision for the value of the threshold value is majorly affected by the values of precision and recall.

Ideally, we want both precision and recall to be 1, but this seldom is the case.

In the case of a Precision-Recall tradeoff, we use the following arguments to decide upon the threshold:

**Low Precision/High Recall:** In a cancer diagnosis application, we do not want any affected patient to be classified as not affected (**False Negative**). Here we want to reduce the number of **false negatives** without necessarily reducing the number of **false positives**. So we choose a **low value of Precision** or a **high value of Recall**.

**High Precision/Low Recall:** We are classifying customers whether they will react positively or negatively to a personalized advertisement. We want to be sure that the customer will react positively to the advertisement because otherwise, a negative reaction can cause a loss of potential sales from the customer. Here we want to **reduce** the **number of false positives** without necessarily reducing the number of false negatives. So, we choose a decision value that has a **high value of Precision** or a **low value of Recall**.

## F1 Score:

To get the best precision and recall at the same time, a metric F1 score is used.

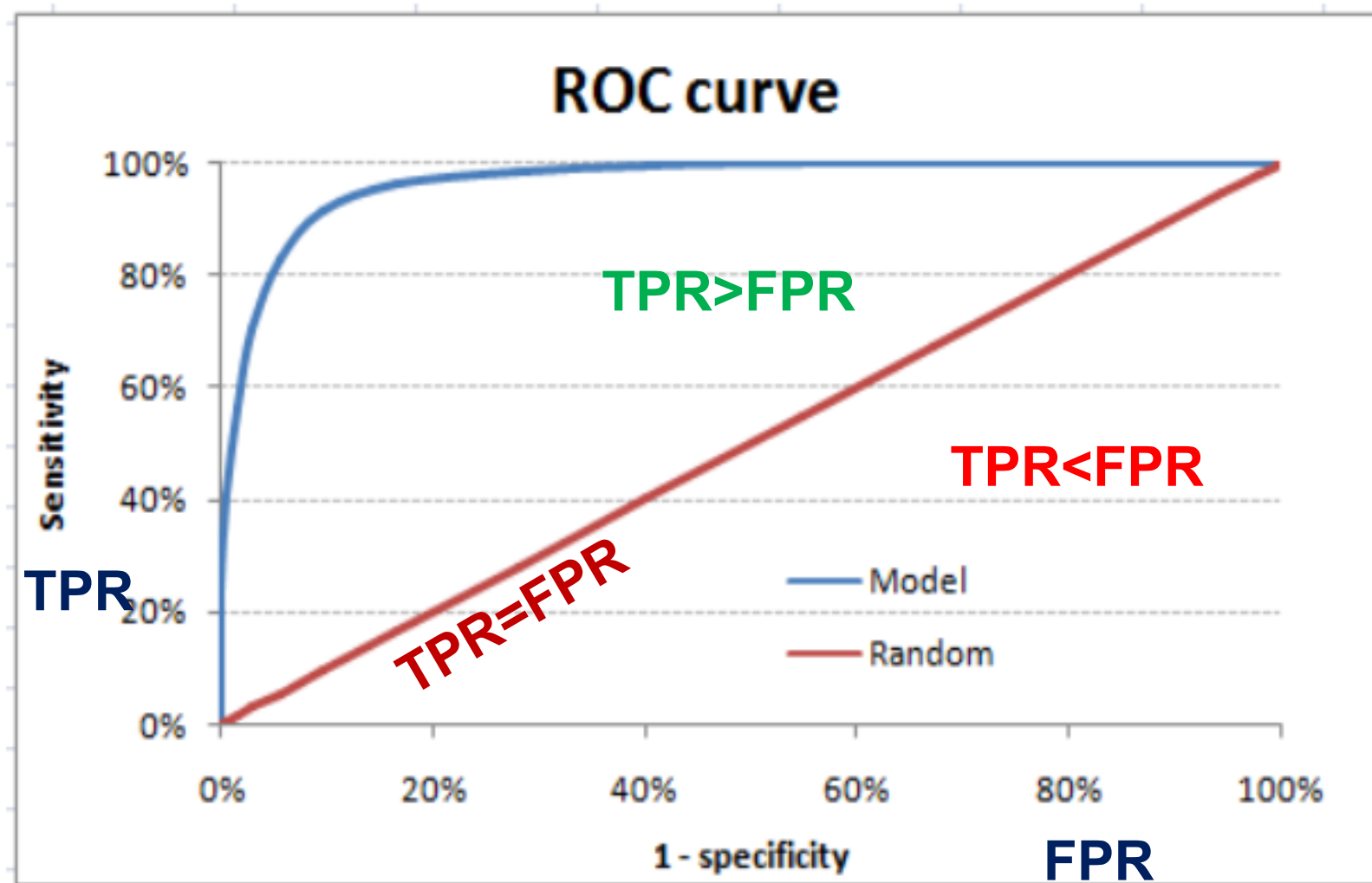
F1-Score is the **harmonic mean of precision and recall** values for a classification problem.

The formula for F1-Score is as follows:

$$\text{F1 score} = \frac{\text{TP}}{\text{TP} + \frac{1}{2}(\text{FP} + \text{FN})}$$

$$F_1 = \left( \frac{\text{recall}^{-1} + \text{precision}^{-1}}{2} \right)^{-1} = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}.$$

# Receiver Operating Characteristics - ROC Curve:



The ROC curve is the plot between True Positive Rate (TPR) and False Positive rate (FPR)

**TPR=TP/(TP+FN)**. It is also known as **Sensitivity** or **Recall**

**FPR=FP/(TP+FN)**. It is **(1- Specificity)**

As both TPR and FPR range between 0 to 1 (or 0% to 100%), the area will always lie between 0 and 1, and a greater value of AUC (area Under Curve) denotes better model performance. Our main goal is to maximize this area in order to have the **highest TPR and lowest FPR** at the given threshold.

## Gini Coefficient

Gini is nothing but the ratio between the area between the ROC curve and the diagonal line i.e. the area of the above triangle.

$Gini = 2 * AUC - 1$  ; Gini above 60% is a good model.

## Log Loss:

AUC ROC only takes into account the order of probabilities, and hence it does not take into account the model's capability to predict a higher probability for samples more likely to be positive.

In that case, we could use the log loss, which is nothing but a negative average of the log of corrected predicted probabilities for each instance.

$$-\frac{1}{N} \sum_{i=1}^N y_i \cdot \log(p(y_i)) + (1 - y_i) \cdot \log(1 - p(y_i))$$

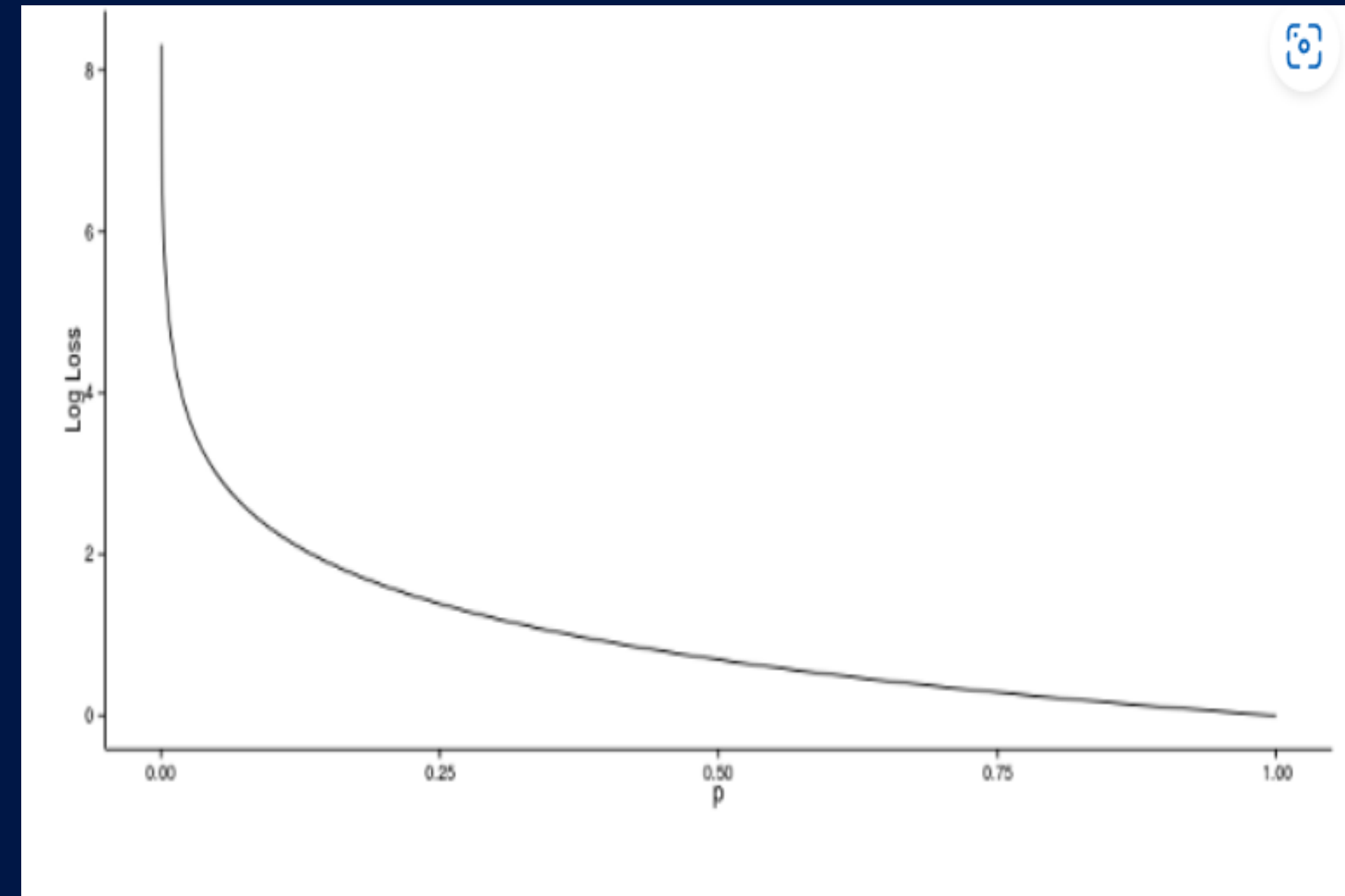
$p(y_i)$  is the predicted probability of a positive class  
 $1-p(y_i)$  is the predicted probability of a negative class  
 $y_i = 1$  for the positive class and 0 for the negative class (actual values)

Let us calculate log loss for a few random values to get the gist of the above mathematical function:

$$\text{Log loss}(1, 0.1) = 2.303$$

$$\text{Log loss}(1, 0.5) = 0.693$$

$$\text{Log loss}(1, 0.9) = 0.105$$



# Root Mean Squared Logarithmic Error (RMSLE)

In the case of Root mean squared logarithmic error, we take the log of the predictions and actual values.

RMSLE is usually used when we don't want to penalize huge differences in the predicted and the actual values when both predicted and true values are huge numbers.

The diagram shows two formulas side-by-side. The left formula is for Root Mean Squared Error (RMSE): 
$$\sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$
 The right formula is for Root Mean Squared Log Error (RMSLE): 
$$\sqrt{\frac{1}{n} \sum_{i=1}^n (\log(p_i + 1) - \log(a_i + 1))^2}$$
 A dashed orange box labeled 'prediction' highlights the terms  $\hat{y}_i$  in the RMSE formula and  $\log(p_i + 1)$  in the RMSLE formula. A dashed purple box labeled 'actual' highlights the terms  $y_i$  in the RMSE formula and  $\log(a_i + 1)$  in the RMSLE formula.

$P_i$  - the prediction of target

$A_i$  - is the actual target for  $i$

- 1.If both predicted and actual values are small: RMSE and RMSLE are the same.
- 2.If either predicted or the actual value is big:  $RMSE > RMSLE$
- 3.If both predicted and actual values are big:  $RMSE > RMSLE$  (RMSLE becomes almost negligible)

THANK YOU

THANK YOU